

Data Science Challenge — Truthfulness

Two parts: build LLM truthfulness models, then deploy them as a live, multi-agent service.

This challenge has two parts that build on each other:

- **Part 1 — Truthfulness Models:** build LLM-based predictors and an explainer for a binary truthfulness classification task.
- **Part 2 — Multi-Agent, Deployed Service:** turn those components into autonomous agents that communicate over A2A, use tools over MCP, and run as live services in the cloud.

Part 2 reuses the endpoints you build in Part 1, so the two parts form a single, coherent body of work. The dataset below is shared across both parts.

The Dataset

The dataset for this challenge consists of thousands of statements (quotes) made by politicians and other public figures. The data includes various information on each statement and on the person who made it. The truthfulness of each statement was also evaluated by expert human editors, who then assigned an appropriate truthfulness label.

All the data is included in the provided `data.csv` file. The file includes one comma-separated line for each statement, with the following attributes (columns):

- **label:** a truthfulness label assigned by human annotators. Possible values (from most to least truthful) are: `true`, `mostly-true`, `half-true`, `barely-true`, `false`, `extremely-false`.
- **statement:** the text of the actual statement that is being evaluated.
- **subjects:** a list of topics related to the statement, separated by a `$` sign — for example `state-financestaxes`.
- **speaker_name:** the name of the person who made the statement.
- **speaker_job:** the job title of the person who made the statement.
- **speaker_state:** the US state associated with the person who made the statement.
- **speaker_affiliation:** the political or professional affiliation of the person — for example `democrat`, `republican`, `journalist`.
- **statement_context:** the context in which the statement was made — for example `a radio interview`, `a press conference`.

The task is a **binary truthfulness classification** problem: each statement should ultimately be predicted as either **True** or **False**. It is up to you to decide how to map the six-way human label onto this binary target.

Truthfulness Models

DELIVERABLE — PYTHON PACKAGE

Part 1 is delivered as a **Python package** that implements the components below. While it is up to you to decide how to structure your package, it should include at least the following.

Common requirements for all components. Each component must be able to **efficiently process a set of data points** (not just one) and return its outputs for all of them. In addition, for `predict()` and `explain()` endpoints, if the caller also supplies the **correct labels** for those points, the component must return **appropriate performance metrics** for its own predictions on that set.

1 Zero-shot LLM Predictor

Build a **pure-LLM predictor** that leverages the prior knowledge of an LLM to predict whether a statement is True or False. It simply prompts the LLM with the attributes of a given row and asks for an answer. **No fine-tuning and no training of any kind** at this stage.

ENDPOINT `predict(points, labels=None)`

- **Input:** `points` — a set of statements with their attributes (excluding the label); optional `labels` — the correct labels for those points.
- **Output:** a True/False prediction for each point. If `labels` are provided, it additionally returns performance metrics for its predictions on the set.

Prompt design, choice of model, and how much of the row's metadata you include are up to you.

2 Fine-tuned LLM Predictor

Fine-tune the LLM-based predictor to improve on the zero-shot baseline. This component has two endpoints — one to perform the fine-tuning and one for inference.

ENDPOINT 1 `fine_tune(training_dataset)`

- **Input:** `training_dataset` — a dataset of statements in the same format as `data.csv` (i.e. including labels).
- **Behavior:** performs the data preparation and train/validation split and runs the fine-tuning, producing a fine-tuned model (the heavy compute may run via a provider's fine-tuning API).
- **Output:** the resulting fine-tuned model, or a handle/identifier for it.

ENDPOINT 2 `predict(points, labels=None)`

- The **same signature and behavior as Component 1's `predict`**, but backed by the fine-tuned model, so the two predictors are interchangeable.

3 Explainer

Build an **explainer** that can work with **either** model (zero-shot or fine-tuned).

ENDPOINT `explain(model, points, labels=None)`

- **Input:** `model` — a predictor (Component 1 or Component 2); `points` — a set of statements with their attributes; optional `labels` — the correct labels for those points.
- **Output:** for each point, the **model's prediction** and an **explanation of the factors that drive that prediction**. If `labels` are provided, it additionally returns performance metrics for the underlying model's predictions on the set.

The nature of the explanations is up to you.

README & Evaluation

Alongside the package code, Part 1 must include a README and a way to reproduce your results:

- **A README** that documents your key decisions and how to run everything:
 - **Target framing:** how you mapped the six-way human label onto the binary True/False target, and why.
 - **Evaluation methodology:** which metrics you chose and why (consider, for example, class balance, calibration, and the cost of different error types), and how you form the held-out split used to compare the two predictors fairly.
 - **Fine-tuning approach:** data preparation, train/validation split, and the key choices that affect the result.
 - **Explainer:** the nature of your explanations and why they are meaningful and trustworthy for this kind of model.
 - **Install and usage instructions.**
- **A reproducible evaluation entrypoint:** a single documented command (for example, `python -m truthfulness.evaluate --data data.csv`) that loads a held-out portion of the data, runs **both** the zero-shot and fine-tuned predictors over it, and prints the chosen metrics for each **side by side**. It should use a fixed config/seed so the numbers are stable when re-run. This builds on the common metrics behavior above so the reported results can be reproduced with one command, rather than relying on quoted numbers.

Part 1 Evaluation Criteria

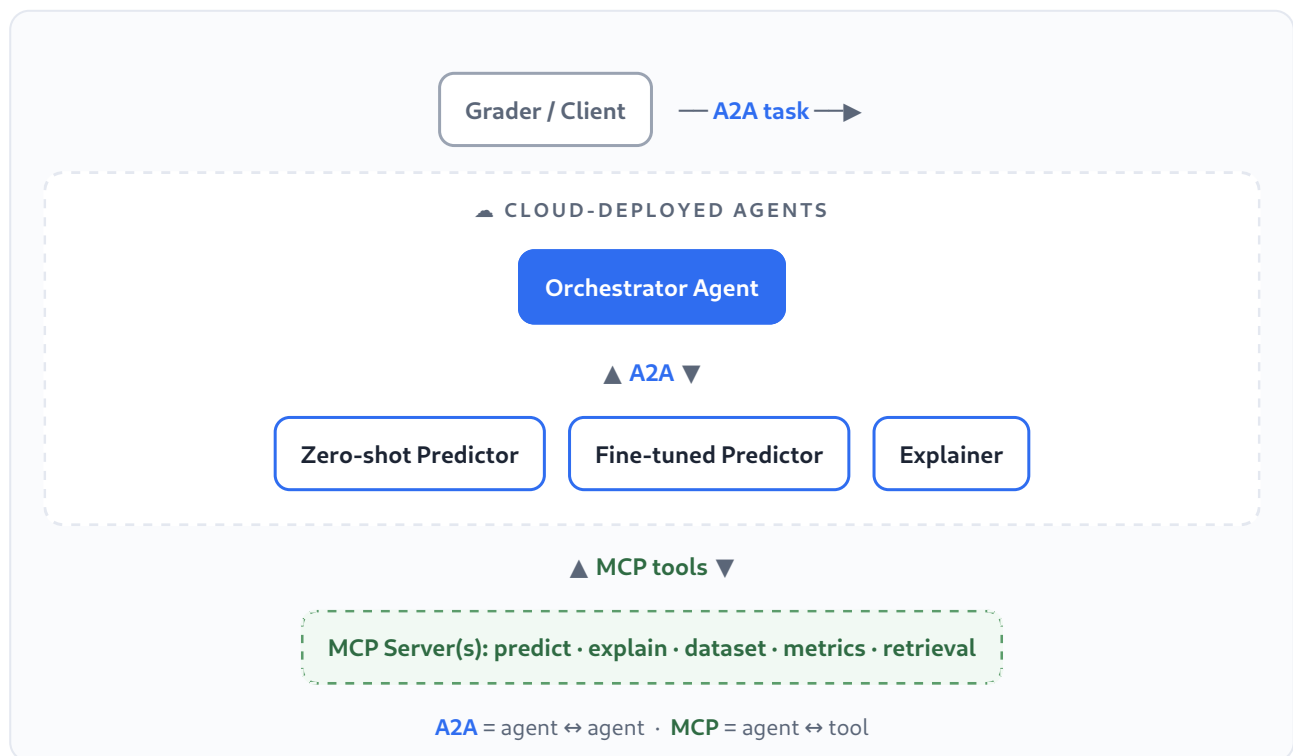
- **Predictive performance:** how well both predictors score on a held-out set under the metrics you chose.
- **A real fine-tuning gain:** the fine-tuned model measurably beats the zero-shot baseline on data it was not trained on — a gain large enough not to be run-to-run noise, with no train/test leakage.
- **Faithful explanations:** each explanation reflects the factors that actually drove that specific prediction and is consistent with the predicted label, rather than generic boilerplate that would fit any statement.
- **Sound evaluation methodology:** appropriate metrics and a fair, leakage-free baseline-vs-fine-tuned comparison on the same held-out split.
- **Reproducibility and clarity:** a README that justifies your decisions and an evaluation entrypoint that regenerates your reported metrics from a clean clone.
- **Code quality:** clarity, structure, and efficiency.

Multi-Agent, Deployed Service

This part assumes Part 1 is complete and reuses its three endpoints: `predict()`, `fine_tune()`, and `explain()`. Your task is to turn those endpoints into a **deployed, multi-agent system** that verifies the truthfulness of statements end to end. The components stay the same; what changes is that they now run as autonomous agents that talk to each other over **A2A**, consume capabilities as tools over **MCP**, and live as **services deployed to the cloud** that can be called directly.

What You're Building

A small network of agents, each wrapping one of your Part 1 capabilities, coordinated by an orchestrator:



Two distinct kinds of communication are required, and both will be assessed:

- **A2A (agent-to-agent):** how your agents talk to each other. Each agent advertises its capabilities (an agent card / capability descriptor), and the orchestrator **delegates tasks** to the predictor and explainer agents over A2A, then aggregates their results. This is more than internal HTTP calls — agents should discover and invoke one another as first-class A2A participants.
- **MCP (agent-to-tool):** how an agent uses a capability or tool. Your underlying capabilities are exposed as **MCP tools** and consumed by the agents as MCP clients. Because they are MCP tools, they are reusable by any MCP-compatible client (e.g. Claude or Cursor), not only by your own agents.

In short: A2A is between agents; MCP is between an agent and its tools. A strong submission makes that separation clear in both the design and the running system.

Part 2 Deliverable 1 — Agents & Protocols

DELIVERABLE 1 — MULTI-AGENT SYSTEM

Wrap your Part 1 endpoints as agents and wire them together.

Required agents

- **Orchestrator Agent** — the entry point. Receives a set of statements (and optional labels), coordinates the other agents via A2A, and returns the consolidated result.
- **Zero-shot Predictor Agent** — wraps Component 1's `predict()`.
- **Fine-tuned Predictor Agent** — wraps Component 2's `predict()` (backed by the fine-tuned model).
- **Explainer Agent** — wraps Component 3's `explain()`.
- **(Optional) Fine-tune Agent** — wraps Component 2's `fine_tune()` as an asynchronous training job.

A2A requirements

- **Capability discovery:** each agent publishes an agent card / capability descriptor so others can find out what it does and how to call it.
- **Task delegation:** the orchestrator hands work to the predictor and explainer agents over A2A rather than calling functions in-process.
- **Aggregation / consensus:** the orchestrator combines the agents' outputs into a single answer per statement, including a simple **reconciliation** strategy for when the zero-shot and fine-tuned predictors disagree (your choice of strategy — e.g. defer to the fine-tuned model, use the explainer, or a confidence rule).

MCP requirements

- Expose **at least** the `predict` and `explain` capabilities as **MCP tools** served by one or more MCP servers.
- Expose **at least one shared supporting tool** as MCP as well — for example dataset access, metric computation, or a fact-check / retrieval tool.
- Agents must **consume** these tools as MCP clients. Demonstrate that the tools also work when called from a generic MCP client, not just from your agents.

End-to-end behavior

The deployed Orchestrator must accept a **set** of statements and run the full flow: fan out to the predictor agents via A2A, obtain explanations from the explainer agent, and return — **for each statement** — a final True/False verdict and an explanation. When the caller also supplies the **correct**

labels, the orchestrator additionally returns **aggregate performance metrics** for the run (consistent with Part 1's `predict()` / `explain()` behavior).

An illustrative request / response shape (yours may differ):

```
// Request to the Orchestrator
{
  "points": [
    {
      "statement": "Our state added 50,000 jobs last quarter.",
      "subjects": "state-finance$jobs",
      "speaker_name": "Jane Doe",
      "speaker_job": "Governor",
      "speaker_state": "Ohio",
      "speaker_affiliation": "democrat",
      "statement_context": "a press conference"
    }
  ],
  "labels": ["mostly-true"]
}
```

```
// Response from the Orchestrator
{
  "results": [
    {
      "prediction": true,
      "agreement": { "zero_shot": true, "fine_tuned": true },
      "explanation": "Both predictors agreed; the claim is specific and consistent with the speaker's prior record..."
    }
  ],
  "metrics": { "accuracy": 1.0, "f1": 1.0, "n": 1 }
}
```

Part 2 Deliverable 2 — Cloud Deployment (Live)

DELIVERABLE 2 — LIVE IN THE CLOUD

By the end of Part 2 the whole agent network should be **running in the cloud, not on your laptop**, and it should be possible to exercise it externally: send a batch of statements to a public URL and get verdicts, explanations, and metrics back. A demo on your machine is not enough — the deployment will be called directly during review.

Concretely, the deployment should provide:

- **Containerized services.** Package each agent (and each MCP server) as a container image so it can run anywhere without your local setup. Aim for one deployable unit per agent rather than a single monolith, so the A2A boundaries between agents are real and not just in-process function calls.
- **Infrastructure as Code.** Stand the whole system up from **declarative IaC — Terraform is the preferred default**, but equivalents are perfectly fine (Pulumi, AWS CDK, a Helm chart, or provider-

native declarative config such as Cloud Run YAML). The goal is that the entire deployment can be recreated from scratch with a few commands, without clicking around a cloud console; a purely imperative `deploy.sh` of CLI calls is discouraged. Any cloud provider is fine — the priority is reproducibility and a reachable endpoint, not the vendor.

- **A live, public endpoint.** Deploy the Orchestrator behind a public URL and keep it running through the review period. Provide the URL and anything required to call it (the auth key and the expected request shape). For example, the deployment should be verifiable with a single request:

```
curl -X POST https://<your-orchestrator-url>/verify \
  -H "Authorization: Bearer <your-key>" \
  -H "Content-Type: application/json" \
  -d @batch.json
```

where `batch.json` is a set of statements in the shape shown above. When the correct labels are included, the response should also carry aggregate metrics.

- **Sensible security.** Don't leave the endpoint wide open. A simple API key or bearer token (passed in a header) is enough — just document how to supply it.
- **Observability.** Emit logs/traces that make it possible to follow a single request as it fans out across the agents — for example a request or trace ID that ties the Orchestrator call to the predictor and explainer hops. This is what makes the multi-agent flow debuggable when something misbehaves.

Part 2 Evaluation Criteria

- **Multi-agent flow:** correctness and robustness of the end-to-end orchestration, including how disagreements between predictors are handled.
- **A2A usage:** genuine agent-to-agent communication (capability discovery + task delegation), not just internal HTTP calls.
- **MCP usage:** correct exposure of capabilities as MCP tools and correct consumption of them as MCP clients.
- **Deployment quality:** reproducible IaC, a secured live endpoint, and sensible observability.
- **Code quality:** clarity, structure, and efficiency, building cleanly on the Part 1 package.

How to Submit

Share your work as a **private Git repository** and email a link or invitation to the same email address from which you received the challenge. For Part 2, also include the **live Orchestrator URL and any credentials** required to call it.

Sharing Your Submission

Use a **private Git repository** (for example on GitHub or GitLab) and grant access to the reviewers. Keep it private — do not publish the challenge or your code to a public repository.

Please make sure the repository:

- **Runs from a clean clone:** the README's install steps and the single evaluation command work without extra setup.
- **Pins dependencies:** include a lockfile (for example `uv.lock` or `requirements.txt`) and state the Python version.
- **Contains no secrets:** never commit API keys; read them from the environment and provide a `.env.example` .
- **Excludes bulk artifacts:** do not commit the dataset, fine-tuned weights, or caches — reference how to obtain or regenerate them, and keep a sensible `.gitignore` .

This challenge is meant to demonstrate the way you approach the problem and lay the foundations for a solid solution. A perfect solution is not expected :-). Do your best in the time that you have, and feel free to reach out with questions at any point!